

HDOC DAY-20

1. Problem Statement

Develop, compile, and execute a menu-driven modular C project that calculates geometrical measurements (surface area, volume, base area) and physical properties (mass, weight, potential energy, pressure, buoyant force, buoyancy behavior) of selected solids (Sphere, Cylinder, Cone, Cuboid). The program strictly adheres to modular constraints without using recursion, arrays, pointers, structures, unions, enumerations, typedef, or `<math.h>` functions.

2. Modular Organization

Module	Responsibility / Functions Defined
solid.h	Contains header guards, symbolic constants (PI=3.1416, GRAVITY=9.8, TOLERANCE=0.000001, choices, buoyancy results), and external function prototypes.
math_utils.c	Calculates square root iteratively using Newton-Raphson method (calculate_square_root()).

Module	Responsibility / Functions Defined
geometry.c	Calculates surface area, volume, base area, and slant height for Sphere, Cylinder, Cone, and Cuboid.
physics.c	Obtains solid dimensions and performs calculations for mass, weight, potential energy, pressure, buoyant force, and buoyancy state.
main.c	Displays interactive main menu, validates input values, invokes functions, and displays formatted results in SI units.

3. Algorithms

Module: math_utils.c

- **calculate_square_root(number):** If $\text{number} < 0$ return -1; if $\text{number} == 0$ return 0. Set $\text{old_value} = \text{number}$. In a do-while loop, calculate $\text{new_value} = 0.5 * (\text{old_value} + \text{number} / \text{old_value})$. Compute absolute difference $|\text{new_value} - \text{old_value}|$. Continue while $\text{difference} \geq 0.000001$. Return new_value .

Module: geometry.c

- **sphere_area(radius):** Return $4 \cdot \pi \cdot r^2$.
- **sphere_volume(radius):** Return $(4/3) \cdot \pi \cdot r^3$.
- **cylinder_area(radius, height):** Return $2 \cdot \pi \cdot r \cdot (r + h)$.
- **cylinder_volume(radius, height):** Return $\pi \cdot r^2 \cdot h$.
- **cone_area(radius, height):** Calculate slant height $l = \text{calculate_square_root}(r^2 + h^2)$. Return $\pi \cdot r \cdot (r + l)$.
- **cone_volume(radius, height):** Return $(1/3) \cdot \pi \cdot r^2 \cdot h$.
- **cuboid_area(length, breadth, height):** Return $2 \cdot (l \cdot b + b \cdot h + l \cdot h)$.
- **cuboid_volume(length, breadth, height):** Return $l \cdot b \cdot h$.
- **circular_base_area(radius):** Return $\pi \cdot r^2$.
- **cuboid_base_area(length, breadth):** Return $l \cdot b$.

Module: physics.c

- **calculate_mass(density):** Get solid type, calculate volume V , return density $\cdot V$.
- **calculate_weight(density):** Calculate mass m , return $m \cdot g$.
- **potential_energy(density, elevation):** Calculate mass m , return $m \cdot g \cdot h$.

- **calculate_pressure(density):** Get solid type (cylinder/cone/cuboid). Determine contact base area A and volume V . Return $(\text{density} \cdot V \cdot g)/A$.
- **buoyant_force(fluid_density):** Get solid volume V , return $\text{fluid_density} \cdot V \cdot g$.
- **determine_buoyancy(solid_density, fluid_density):** If solid density $>$ fluid density return SINKS (1). If solid density $<$ fluid density return FLOATS (-1). If equal return SUSPENDED (0).

4. Test Case Table

Test Case	Option	Input Parameters	Expected Output	Actual Output	Status
Input Validation	Any	Radius = -5 → 3	"Value must be greater than zero. Try again."	Prompt error and re-request value	Pass
Sphere Area	1	Solid: Sphere, Radius: 2	50.2656 m ²	50.2656 m ²	Pass

Test Case	Option	Input Parameters	Expected Output	Actual Output	Status
Cylinder Volume	2	Solid: Cylinder, Radius: 2, Height: 5	62.8320 m ³	62.8320 m ³	Pass
Base Area Check	3	Solid: Sphere	"Sphere does not have a defined base area!"	Error prompt displayed	Pass
Base Area Cuboid	3	Solid: Cuboid, Length: 4,	12.0000 m ²	12.0000 m ²	Pass

Test Case	Option	Input Parameters	Expected Output	Actual Output	Status
		Breadth: 3			
Mass Calculation	4	Density : 1000, Solid: Sphere (r=1)	4188.8000 kg	4188.8000 kg	Pass
Potential Energy	6	Density : 500, Elevation: 10, Solid: Cuboid (2x2x2)	392000.0000 J	392000.0000 J	Pass
Pressure Test	7	Density : 1000, Solid: Cylinder	49000.0000 Pa	49000.0000 Pa	Pass

Test Case	Option	Input Parameters	Expected Output	Actual Output	Status
		r (r=2, h=5)			
Buoyancy (Float)	9	Solid Density : 800, Fluid Density : 1000	Solid FLOATS on the fluid.	Solid FLOATS on the fluid.	Pass
Buoyancy (Sink)	9	Solid Density : 2700, Fluid Density : 1000	Solid SINKS in the fluid.	Solid SINKS in the fluid.	Pass

5. Source Code Listings

solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
```

```

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double calculate_square_root(double number);

double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

double calculate_mass(double density);
double calculate_weight(double density);
double potential_energy(double density, double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

#endif

```

math_utils.c

```

#include "solid.h"

double calculate_square_root(double number) {
    if (number < 0) return -1.0;
    if (number == 0) return 0.0;

```

```
double old_value = number;
double new_value;
double difference;

do {
    new_value = 0.5 * (old_value + number / old_value);
    difference = new_value - old_value;
    if (difference < 0) {
        difference = -difference;
    }
    old_value = new_value;
} while (difference >= TOLERANCE);

return new_value;
}
```

geometry.c

```
#include "solid.h"

double sphere_area(double radius) {
    return 4.0 * PI * radius * radius;
}

double sphere_volume(double radius) {
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height) {
    return 2.0 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height) {
    return PI * radius * radius * height;
}

double cone_area(double radius, double height) {
```

```

    double slant_height = calculate_square_root(radius * radius + height *
height);
    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height) {
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height) {
    return 2.0 * (length * breadth + breadth * height + length * height);
}

double cuboid_volume(double length, double breadth, double height) {
    return length * breadth * height;
}

double circular_base_area(double radius) {
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth) {
    return length * breadth;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

static int select_solid_type(void) {
    int choice;
    do {
        printf("\nSelect Solid Type:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice (1-4): ");
        scanf("%d", &choice);
        if (choice < 1 || choice > 4) {
            printf("Invalid selection! Try again.\n");

```

```

    }
    } while (choice < 1 || choice > 4);
    return choice;
}

static double get_positive_input(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) {
            printf("Value must be greater than zero. Try again.\n");
        }
    } while (val <= 0);
    return val;
}

static double get_solid_volume(int solid_type) {
    double vol = 0.0;
    if (solid_type == SPHERE) {
        double r = get_positive_input("Enter radius (m): ");
        vol = sphere_volume(r);
    } else if (solid_type == CYLINDER) {
        double r = get_positive_input("Enter radius (m): ");
        double h = get_positive_input("Enter height (m): ");
        vol = cylinder_volume(r, h);
    } else if (solid_type == CONE) {
        double r = get_positive_input("Enter radius (m): ");
        double h = get_positive_input("Enter height (m): ");
        vol = cone_volume(r, h);
    } else if (solid_type == CUBOID) {
        double l = get_positive_input("Enter length (m): ");
        double b = get_positive_input("Enter breadth (m): ");
        double h = get_positive_input("Enter height (m): ");
        vol = cuboid_volume(l, b, h);
    }
    return vol;
}

double calculate_mass(double density) {

```

```

int type = select_solid_type();
double volume = get_solid_volume(type);
return density * volume;
}

double calculate_weight(double density) {
    double mass = calculate_mass(density);
    return mass * GRAVITY;
}

double potential_energy(double density, double elevation) {
    double mass = calculate_mass(density);
    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    int choice;
    do {
        printf("\nSelect Solid Type for Pressure (Sphere not applicable):\n");
        printf("2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice (2-4): ");
        scanf("%d", &choice);
        if (choice < 2 || choice > 4) {
            printf("Invalid selection! Try again.\n");
        }
    } while (choice < 2 || choice > 4);

    double contact_area = 0.0, volume = 0.0;

    if (choice == CYLINDER) {
        double r = get_positive_input("Enter radius (m): ");
        double h = get_positive_input("Enter height (m): ");
        contact_area = circular_base_area(r);
        volume = cylinder_volume(r, h);
    } else if (choice == CONE) {
        double r = get_positive_input("Enter radius (m): ");
        double h = get_positive_input("Enter height (m): ");
        contact_area = circular_base_area(r);
        volume = cone_volume(r, h);
    } else if (choice == CUBOID) {

```

```

    double l = get_positive_input("Enter length (m): ");
    double b = get_positive_input("Enter breadth (m): ");
    double h = get_positive_input("Enter height (m): ");
    contact_area = cuboid_base_area(l, b);
    volume = cuboid_volume(l, b, h);
}

return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density) {
    int type = select_solid_type();
    double volume = get_solid_volume(type);
    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (solid_density > fluid_density) {
        return SINKS;
    } else if (solid_density < fluid_density) {
        return FLOATS;
    } else {
        return SUSPENDED;
    }
}
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

static double get_positive(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) {
            printf("Value must be greater than zero. Try again.\n");
        }
    }
}

```

```

    } while (val <= 0);
    return val;
}

static double get_non_negative(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val < 0) {
            printf("Value cannot be negative. Try again.\n");
        }
    } while (val < 0);
    return val;
}

static int select_solid_menu(void) {
    int choice;
    do {
        printf("\n--- SELECT SOLID ---\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice (1-4): ");
        scanf("%d", &choice);
        if (choice < 1 || choice > 4) {
            printf("Invalid selection! Try again.\n");
        }
    } while (choice < 1 || choice > 4);
    return choice;
}

int main(void) {
    int main_choice;

    do {
        printf("\n===== MAIN MENU =====\n");
        printf("1. Surface Area\n");
        printf("2. Volume\n");
        printf("3. Base Area\n");
        printf("4. Mass\n");
        printf("5. Weight\n");
    }

```

```

printf("6. Gravitational Potential Energy\n");
printf("7. Pressure on the Base\n");
printf("8. Buoyant Force\n");
printf("9. Buoyancy Behavior\n");
printf("10. Exit\n");
printf("Enter choice (1-10): ");
scanf("%d", &main_choice);

switch (main_choice) {
    case 1: {
        int s = select_solid_menu();
        if (s == SPHERE) {
            double r = get_positive("Enter radius (m): ");
            printf("Surface Area of Sphere: %.4lf m^2\n", sphere_area(r));
        } else if (s == CYLINDER) {
            double r = get_positive("Enter radius (m): ");
            double h = get_positive("Enter height (m): ");
            printf("Surface Area of Cylinder: %.4lf m^2\n", cylinder_area(r,
h));
        } else if (s == CONE) {
            double r = get_positive("Enter radius (m): ");
            double h = get_positive("Enter height (m): ");
            printf("Surface Area of Cone: %.4lf m^2\n", cone_area(r, h));
        } else if (s == CUBOID) {
            double l = get_positive("Enter length (m): ");
            double b = get_positive("Enter breadth (m): ");
            double h = get_positive("Enter height (m): ");
            printf("Surface Area of Cuboid: %.4lf m^2\n", cuboid_area(l, b,
h));
        }
        break;
    }
    case 2: {
        int s = select_solid_menu();
        if (s == SPHERE) {
            double r = get_positive("Enter radius (m): ");
            printf("Volume of Sphere: %.4lf m^3\n", sphere_volume(r));
        } else if (s == CYLINDER) {
            double r = get_positive("Enter radius (m): ");
            double h = get_positive("Enter height (m): ");

```

```

        printf("Volume of Cylinder: %.4lf m^3\n", cylinder_volume(r, h));
    } else if (s == CONE) {
        double r = get_positive("Enter radius (m): ");
        double h = get_positive("Enter height (m): ");
        printf("Volume of Cone: %.4lf m^3\n", cone_volume(r, h));
    } else if (s == CUBOID) {
        double l = get_positive("Enter length (m): ");
        double b = get_positive("Enter breadth (m): ");
        double h = get_positive("Enter height (m): ");
        printf("Volume of Cuboid: %.4lf m^3\n", cuboid_volume(l, b, h));
    }
    break;
}
case 3: {
    int s;
    do {
        printf("\n--- SELECT SOLID FOR BASE AREA ---\n");
        printf("2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice (2-4): ");
        scanf("%d", &s);
        if (s < 2 || s > 4) {
            printf("Sphere does not have a defined base area! Select 2, 3,
or 4.\n");
        }
    } while (s < 2 || s > 4);

    if (s == CYLINDER || s == CONE) {
        double r = get_positive("Enter radius (m): ");
        printf("Base Area: %.4lf m^2\n", circular_base_area(r));
    } else if (s == CUBOID) {
        double l = get_positive("Enter length (m): ");
        double b = get_positive("Enter breadth (m): ");
        printf("Base Area of Cuboid: %.4lf m^2\n", cuboid_base_area(l,
b));
    }
    break;
}
case 4: {
    double d = get_positive("Enter solid density (kg/m^3): ");
    printf("Mass: %.4lf kg\n", calculate_mass(d));
}

```

```

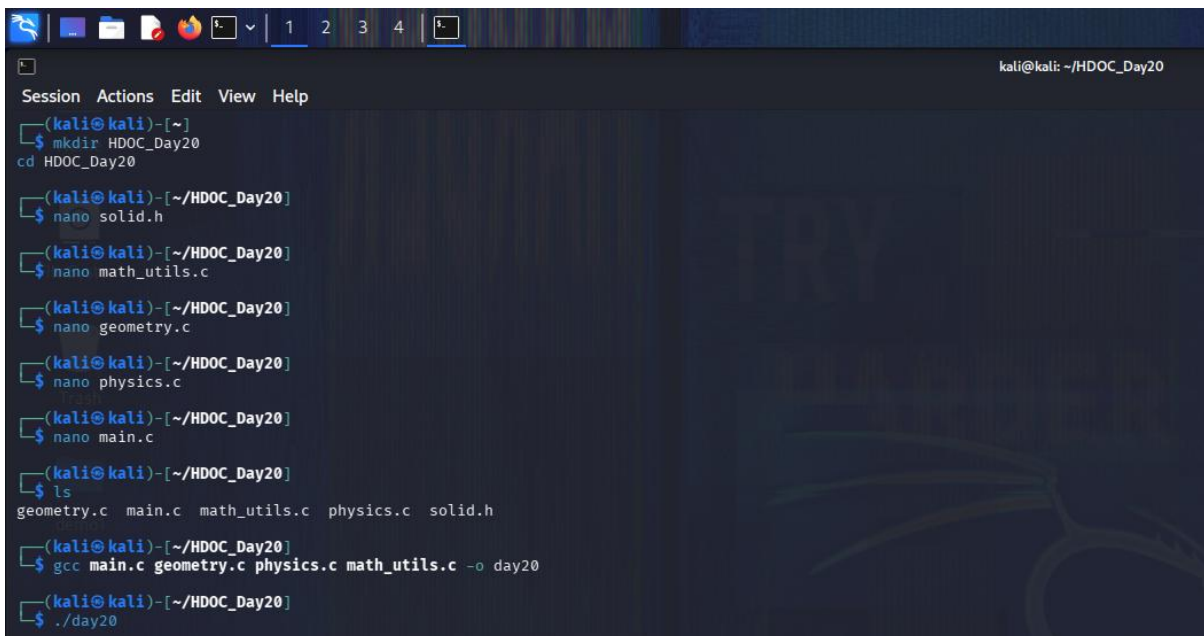
        break;
    }
    case 5: {
        double d = get_positive("Enter solid density (kg/m^3): ");
        printf("Weight: %.4lf N\n", calculate_weight(d));
        break;
    }
    case 6: {
        double d = get_positive("Enter solid density (kg/m^3): ");
        double e = get_non_negative("Enter elevation (m): ");
        printf("Gravitational Potential Energy: %.4lf J\n",
potential_energy(d, e));
        break;
    }
    case 7: {
        double d = get_positive("Enter solid density (kg/m^3): ");
        printf("Pressure on Base: %.4lf Pa\n", calculate_pressure(d));
        break;
    }
    case 8: {
        double fd = get_positive("Enter fluid density (kg/m^3): ");
        printf("Buoyant Force: %.4lf N\n", buoyant_force(fd));
        break;
    }
    case 9: {
        double sd = get_positive("Enter solid density (kg/m^3): ");
        double fd = get_positive("Enter fluid density (kg/m^3): ");
        int res = determine_buoyancy(sd, fd);
        if (res == SINKS) {
            printf("Result: Solid SINKS in the fluid.\n");
        } else if (res == FLOATS) {
            printf("Result: Solid FLOATS on the fluid.\n");
        } else {
            printf("Result: Solid remains SUSPENDED in the fluid.\n");
        }
        break;
    }
    case 10:
        printf("Exiting program...\n");
        break;

```

```
        default:
            printf("Invalid menu choice! Select 1-10.\n");
            break;
    }
} while (main_choice != 10);

return 0;
}
```

6. Execution & Verification Screenshots



```
kali@kali: ~/HDOC_Day20
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir HDOC_Day20
cd HDOC_Day20
(kali@kali)-[~/HDOC_Day20]
$ nano solid.h
(kali@kali)-[~/HDOC_Day20]
$ nano math_utils.c
(kali@kali)-[~/HDOC_Day20]
$ nano geometry.c
(kali@kali)-[~/HDOC_Day20]
$ nano physics.c
(kali@kali)-[~/HDOC_Day20]
$ nano main.c
(kali@kali)-[~/HDOC_Day20]
$ ls
geometry.c  main.c  math_utils.c  physics.c  solid.h
(kali@kali)-[~/HDOC_Day20]
$ gcc main.c geometry.c physics.c math_utils.c -o day20
(kali@kali)-[~/HDOC_Day20]
$ ./day20
```

```
Session Actions Edit View Help
kali@kali - /HDDC_Day20
kali@kali:~/HDDC_Day20$ ./day20

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter choice (1-10): 1

===== SELECT SOLID =====
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice (1-4): 1
Enter radius (m): 2
Surface Area of Sphere: 50.2656 m^2

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter choice (1-10): 9
Enter solid density (kg/m^3): 800
Enter fluid density (kg/m^3): 1000
Result: Solid FLOATS on the fluid.

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter choice (1-10): 10
Exiting program...

kali@kali:~/HDDC_Day20$
```